



Universidad Tecnológica del Perú

“Año de la Esperanza y el Fortalecimiento de la Democracia”

Facultad de Ingeniería de Sistemas e Informática

Diseño de Productos y Servicios

Sistema IA para priorización de emergencias

Bustamante Soria, Fabrizio Alessandro – 100%

Garcia Ortega, Shayuri Kiara- 100%

Mantari Licapa, Walter Isaac – 100%

Ramirez Rivera, Adrian Jose Felix – 100%

Rodriguez Munaylla, Marcela Natalie – 100%

Asistente Virtual: Emergenc- IA – 100%

Guevara Jimenez Jorge Alfredo

2026 – Lima

Tema: Aplicación del Ciclo Deming (PDCA) para la Mejora Continua de un Sistema IA para Priorización de Emergencias en el Distrito de Comas

1. Explicación conceptual del Ciclo Deming (PDCA)

El ciclo Deming o ciclo **Plan-Do-Check-Act (PDCA)** es una metodología de mejora continua aplicada para optimizar procesos, productos y servicios.

Etap	Descripción
PLAN	Identificar problemas y planificar mejoras
DO	Implementar las mejoras
CHECK	Verificar resultados obtenidos
ACT	Estandarizar mejoras y proponer nuevas acciones

2. Objetivo de la práctica

Aplicar el ciclo PDCA para mejorar continuamente una aplicación de software de gestión y priorización de emergencias centralizada que utiliza Inteligencia Artificial para el triaje automatizado por lenguaje natural.

3. Caso de estudio

La aplicación desarrollada permite:

- Registrar citas médicas.
 - Capturar automáticamente la ubicación GPS del ciudadano en riesgo.
 - Reportar emergencias por texto mediante un chatbot interactivo.
 - Clasificar el tipo de incidente y severidad usando Inteligencia Artificial (NLP).
 - Guardar la información consolidada en la base de datos centralizada.
 - Despachar y sincronizar alertas en tiempo real con Serenazgo, PNP y Bomberos.
-

4. Requisitos técnicos de la aplicación

La solución debe implementar arquitectura en capas usando:

Capa	Responsabilidad
Vista	Interfaces de usuario en React (Dashboard y Chatbot)
Controlador	Recepción de solicitudes HTTP mediante Endpoints RESTful
Modelo	Lógica de negocio en Spring Boot y gestión de incidentes
Base de Datos	Persistencia relacional mediante PostgreSQL
Integración IA	Comunicación con la API de Google Gemini

5. Ejemplo de estructura GitHub esperada

Organización de carpetas

```
.github/workflows/
backend/
  src/main/java/.../
    config/
    controller/
    dto/
    entity/
    exception/
    repository/
    service/
  pom.xml
database/
  backups/
  procedures/
  scripts/
docs/
  arquitectura/
  diagramas/
  historias-usuario/
frontend/
  web-app/
    public/
    src/
      assets/
      components/
      context/
      data/
tests/
```

6. Historia de Usuario Principal

Historia de Usuario

Como ciudadano en situación de riesgo.

Necesito registrar la descripción de mi emergencia en el portal o chatbot.

Para obtener una clasificación y priorización inmediata mediante IA antes de que un operador humano procese manualmente mi caso.

7. Criterios de aceptación

Escenario 1 — Registro y envío de reporte

Given que el ciudadano accede al portal web de SafeDistrict o activa el chatbot.

When registra correctamente la descripción de la emergencia y su ubicación.

Then el sistema almacena el incidente en la base de datos centralizada

Escenario 2 — Predicción mediante IA

Given que el ciudadano registra de forma escrita o asistida los detalles del siniestro.

When solicita el envío inmediato del reporte de auxilio.

Then el sistema consulta la IA (Google Gemini API) enviando el texto del incidente

And muestra el resultado predictivo (Tipo de incidente, severidad y nivel de confianza) en la central de monitoreo.

Escenario 3 — Validación de campos

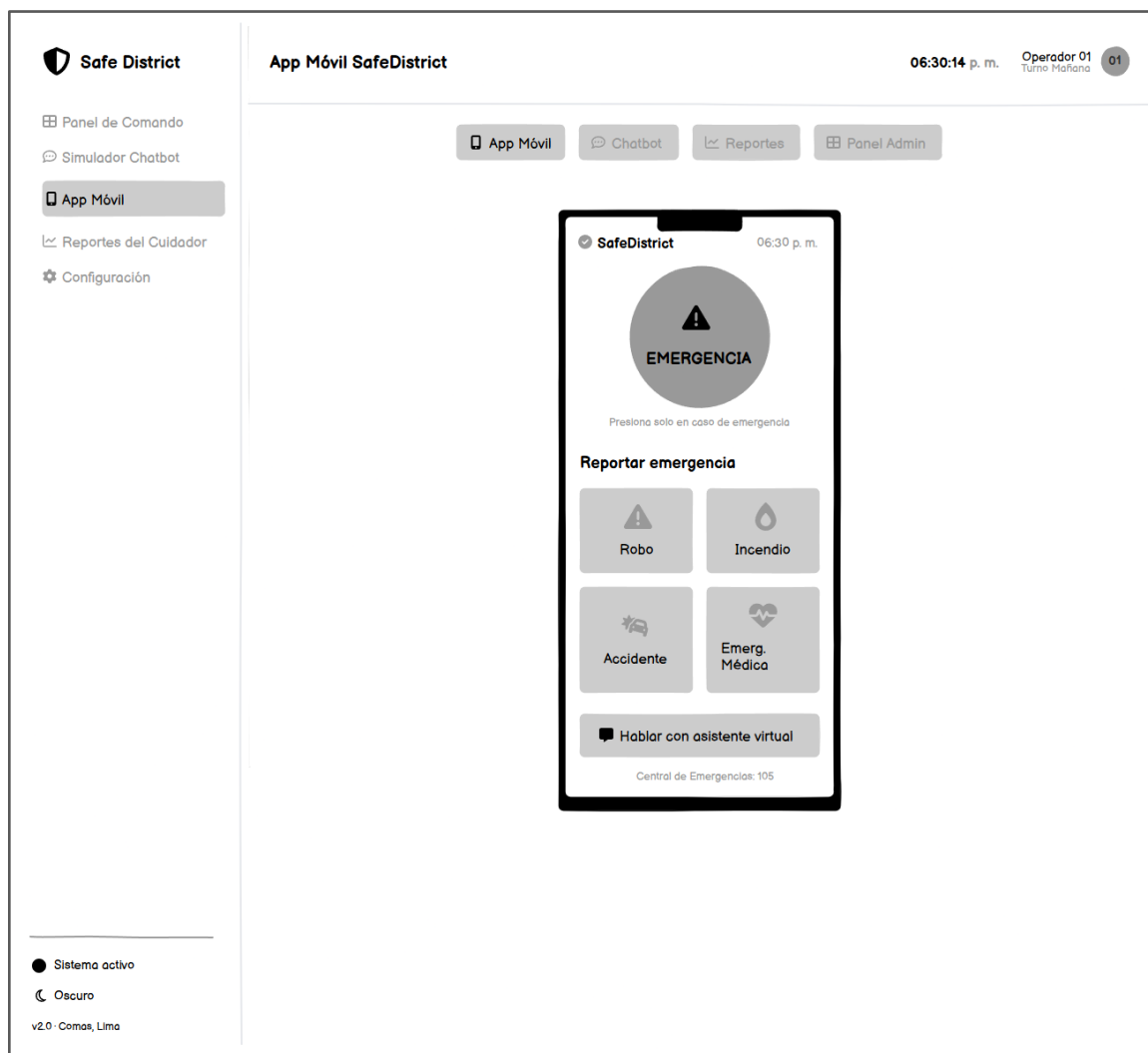
Given que existen campos vacíos o la descripción no cuenta con un mínimo de palabras obligatorias

When el ciudadano intenta enviar el reporte de emergencia

Then el sistema muestra mensajes de validación en la interfaz de usuario para mitigar falsas alertas.

8. Wireframe simplificado

Área	Componentes
Encabezado del celular	Nombre del Sistema
Botón de pánico	Botón de emergencia
Sección de reportes	Título, rejilla de 4 botones con íconos
Acceso a soporte	Botón con ícono de chat IA
Pie de página móvil	Caja de texto



9. Cuadro Integrado de Mejora Continua PDCA (10 ocurrencias)

Dominio	PLAN (Problema)	DO (Mejora)	CHECK (Verificación)	ACT (Acción fija)	Responsable	Fecha
Video explicativo	Desalineado con la HU.	Adaptar guion según la HU.	Validar claridad funcional.	Estandarizar guion técnico.	Shayuri García	02/06/2026
Código GitHub	Carpetas desordenadas.	Reorganizar estructura MVC.	Validar orden de capas.	Mantener estándar modular.	Alessandro Bustamante	01/06/2026
Wireframe	Sin contenedor web/móvil.	Implementar marco responsivo.	Verificar vista móvil.	Reutilizar plantilla de marco.	Marcela Rodríguez	01/06/2026
Wireframe	Ausencia de menú principal.	Añadir menú de navegación (H/V).	Comprobar accesibilidad.	Mantener menú estándar.	Adrian Ramírez	01/06/2026
Wireframe	Ausencia de logo de la app.	Insertar logo de SafeDistrict.	Validar presencia de marca.	Fijar logo en encabezados.	Walter Mantari	01/06/2026
Wireframe	Sin perfil de usuario.	Agregar avatar y rol actual.	Validar datos del operador.	Mantener bloque de sesión.	Marcela Rodríguez	01/06/2026
Wireframe	Sin contenedor de datos/IA.	Crear bloque de triaje e IA.	Verificar lectura de la IA.	Diseñar centrado en IA.	Adrian Ramírez	01/06/2026
Wireframe	Falta de botones inferiores.	Agregar botonera fija.	Probar accesibilidad táctil.	Estandarizar accesos rápidos.	Walter Mantari	01/06/2026
Arquitectura	Lógica en controladores.	Mover lógica a Services.	Validar desacoplamiento.	Aplicar patrón MVC estricto.	Alessandro Bustamante	01/06/2026
Historia Usuario	Escenarios de IA incompletos.	Ampliar criterios con BDD.	Confirmar flujos del chatbot.	Aplicar BDD en toda HU.	Shayuri García	01/06/2026

10. Actividad principal de laboratorio

Indicaciones

Cada equipo deberá:

1. Revisar la arquitectura del proyecto.
 2. Revisar GitHub verificando:
 - Carpetas MVC.
 - Base de datos.
 - Capa integración IA.
 3. Revisar historia de usuario.
 4. Revisar wireframe.
 5. Revisar video demostrativo.
 6. Completar el cuadro PDCA.
 7. Proponer mejoras.
-

11. Lista de verificación de revisión GitHub

Elemento a revisar	Cumple
Carpeta controllers	100%
Carpeta models	100%
Carpeta views	100%
Carpeta database	100%
Carpeta integration	100%
Separación MVC correcta	100%
Clase conexión BD	100%
Clase integración IA	100%
README técnico	100%
Commits organizados	100%

12. Evidencias requeridas

Entregables

- Capturas de GitHub.

SafeDistrict-Diseno Public

main 6 Branches 0 Tags

Go to file Add file Code

Alessandro-BS Arreglo de Chatbot + Actualización README.md 231582e · 2 hours ago 18 Commits

File	Description	Last Commit
.github/workflows	Estructura del proyecto + README.md actualizado	last week
.vscode	Conexión de Dashboard a API	last week
backend	Presentación #1 - Modificar JSON (Chatbot)	last week
database	Estructura del proyecto + README.md actualizado	last week
docs	Estructura del proyecto + README.md actualizado	last week
frontend/web-app	Arreglo de Chatbot + Actualización README.md	2 hours ago
tests	Estructura del proyecto + README.md actualizado	last week
README.md	Arreglo de Chatbot + Actualización README.md	2 hours ago

README

SafeDistrict - Sistema IA para Priorización de Emergencias

SafeDistrict es una solución tecnológica diseñada para optimizar la gestión de emergencias mediante Inteligencia Artificial. El sistema permite la recepción de reportes de ciudadanos, su clasificación automática y la visualización jerárquica de incidentes para los operadores de respuesta.

Este repositorio contiene la arquitectura completa (Full-Stack) de SafeDistrict, integrando un frontend en React, un backend en Spring Boot y procesamiento de Lenguaje Natural (NLP) a través de Google Gemini AI.

Características Principales

1. Dashboard Táctico del Operador (Frontend):

About

[safe-district-diseno.vercel.app](#)

Readme Activity 0 stars 0 watching 0 forks

Releases

No releases published [Create a new release](#)

Deployments 14

Preview last week Production + 12 deployments

Packages

No packages published [Publish your first package](#)

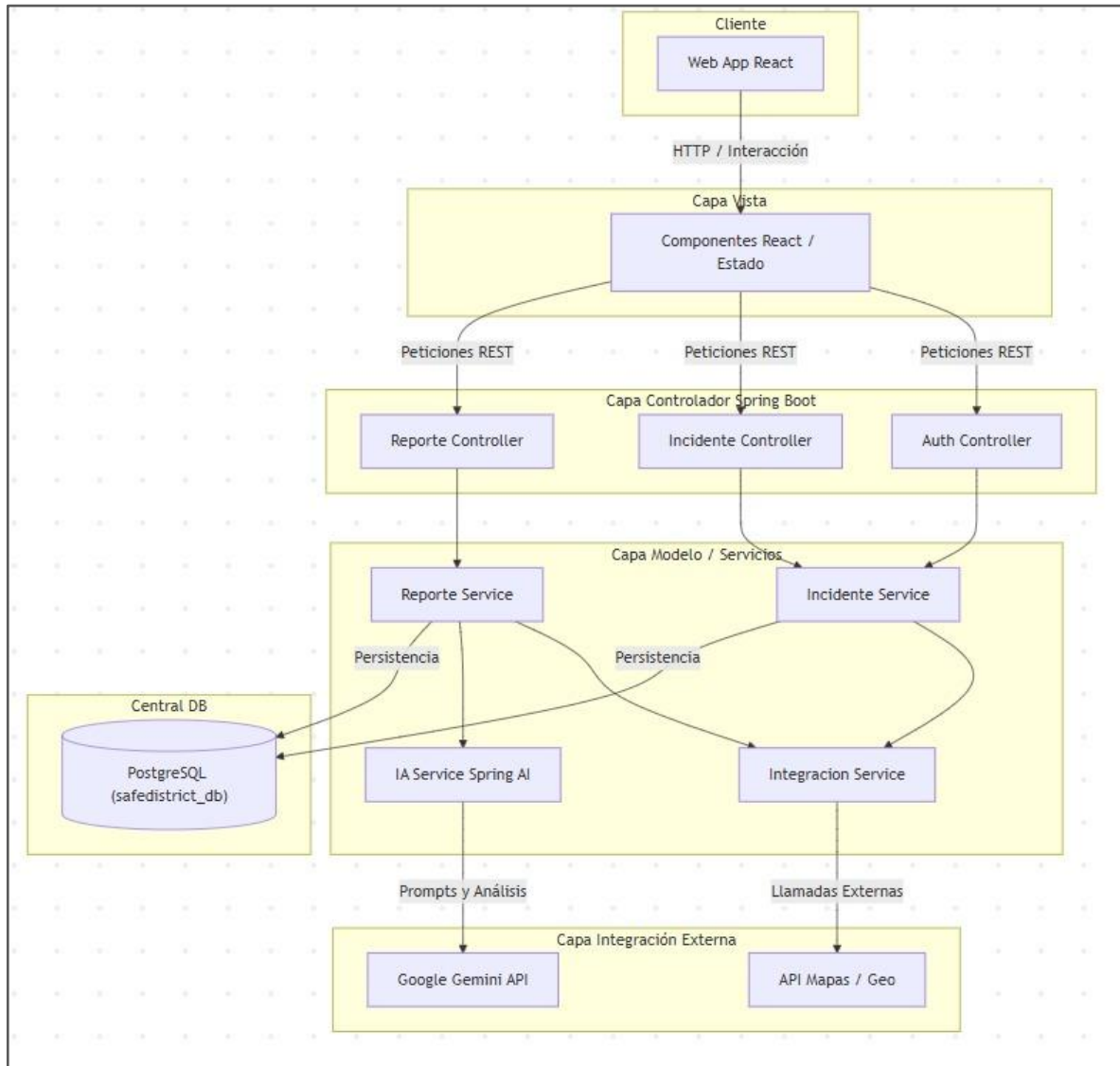
Contributors 2

- Alessandro-BS** Aless Bustamante
- Restring17** Adrian Jose Felix Ramire...

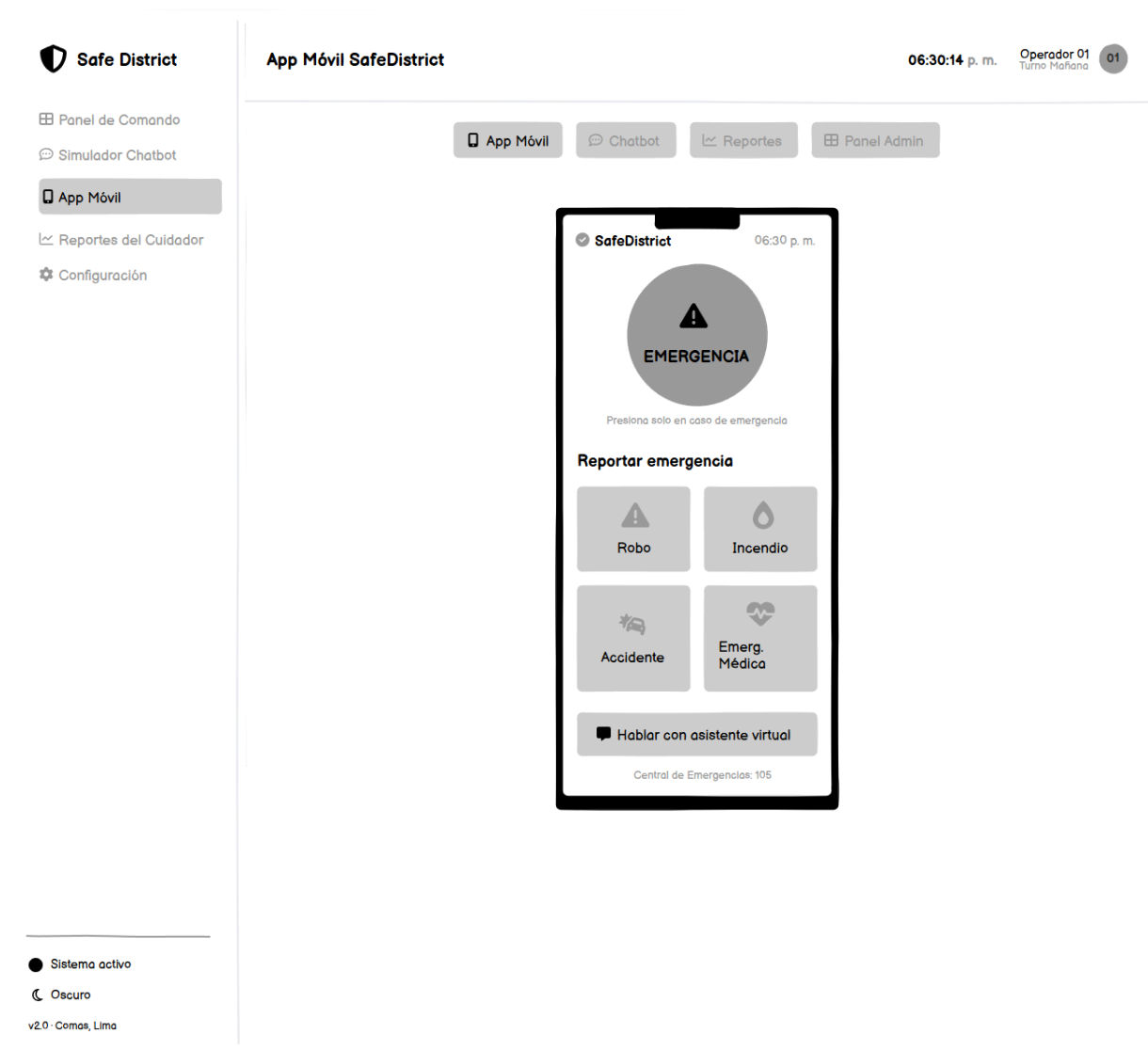
Languages

JavaScript 64.6% Java 28.5%

- Capturas de arquitectura.



- Capturas del wireframe.



Safe District

Panel de Comando

Simulador Chatbot

App Móvil

Reportes del Cuidador

Configuración

Simulador de Reportes Ciudadanos

06:30:14 p. m. Operador 01
Turno Mañana 01

App Móvil

Chatbot

Reportes

Panel Admin

SafeDistrict Assistant

● En línea

¡Hola! Soy el asistente de SafeDistrict. ¿En qué puedo ayudarte hoy?

Reportar emergencia

Estado de mi reporte

Información de seguridad

Reportar emergencia

¿Qué tipo de emergencia necesitas reportar?

Robo

Accidente

Incendio

Emergencia Médica

Robo

Has seleccionado Robo. Describe brevemente lo que está sucediendo (incluye ubicación si es posible):

Robo de celular en Retablo, Comas

Reporte registrado exitosamente

● Sistema activo

🌙 Oscuro

v2.0 · Comas, Lima

- URL del repositorio.
<https://github.com/Alessandro-BS/SafeDistrict-Diseno>
- URL del video funcional.
- Tabla PDCA completa.
- Historia de usuario refinada.

13. Reflexión final

La mejora continua mediante PDCA permite:

- Optimizar la precisión del chatbot basado en la API de Google Gemini para la clasificación de incidentes.
- Garantizar la calidad técnica y mitigar la latencia en la sincronización en tiempo real con Serenazgo, PNP y Bomberos.
- Consolidar buenas prácticas de arquitectura desacoplada estructurando correctamente las capas en React y Spring Boot.
- Mejorar la experiencia del ciudadano ante situaciones de alto estrés mediante interfaces accesibles y de baja fricción.
- Fortalecer el trabajo colaborativo en el equipo de ingeniería para el despliegue ordenado de soluciones tecnológicas de seguridad ciudadana.